

MLCouncil — Piano Operativo Post-AS-IS/TO-BE

2026-05-21

MLCouncil — Piano Operativo Post-AS-IS/TO-BE (2026-05-21)

Executive summary

Stato di partenza (riferimento, 2026-05-21)

Drift register — stato

TO-BE — allineamento con il mio report

Fase 0 — Sync del branch di lavoro con master

Fase 1 — Chiusura drift residui (M2, M3, M5, M8)

Task 1.1 — M5 (residuo): purgare “rolling 100-day IR” da docstrings interni

Task 1.2 — M2: derivare doc risk da config/

Task 1.3 — M3: universo reale documentato

Task 1.4 — M8: separare target engineering da daily inference

Fase 2 — Telemetria fill: prerequisito per cost calibration

Task 2.1 — Schema fill record

Task 2.2 — Hook in OMS e Alpaca adapter

Task 2.3 — Backfill da paper trades storici

Fase 3 — Calibration engine (ADR-0003 Stage B core)

Task 3.1 — Implementation Shortfall computation

Task 3.2 — Manifest + sidecar

Task 3.3 — Job notturno

Fase 4 — Wire calibration in TransactionCostModel

Task 4.1 — Blend lookup + calibrato

Task 4.2 — Lineage in order/execution log

Task 4.3 — Backtest costs walk-forward

Fase 5 — Validation, alerting, promotion

Task 5.1 — Divergence alert

Task 5.2 — Champion/challenger gate (costo, non modello)

Task 5.3 — Baseline post-cost-track

Fase 6 — Dashboard math-trace MVP (P1 observability)

Task 6.1 — Constraint waterfall per il portfolio

Task 6.2 — Council attribution trace

Task 6.3 — Fill quality panel

Fase 7 — Selezione del prossimo track avanzato

Task 7.1 — Mini-spike HRP

Task 7.2 — Mini-spike Robust Optimization

Task 7.3 — Decision review

Riassunto dipendenze tra fasi
Acceptance criteria globale
Comandi rapidi per l'agente
Note operative per l'agente

MLCouncil — Piano Operativo Post-AS-IS/TO-BE (2026-05-21)

Executive summary

Questo piano nasce **dopo** l'ispezione AS-IS / TO-BE prodotta in docs/codebase_analysis.md ed è riconciliato con lo stato attuale di origin/master (HEAD 21b63af). Il team ha già chiuso 8 degli 11 mismatch del drift register (docs/architecture-as-is-to-be-2026-05-21.md) e ha selezionato come **prima traccia avanzata** il Self-Calibrating Cost Model (ADR docs/adr/2026-05-21-self-calibrating-cost-model.md).

Le 7 fasi qui sotto sono progettate per essere eseguite **in sequenza da un agente**: ogni fase ha file target espliciti, comandi di verifica, messaggio di commit e definizione di "fatto". Le fasi 0-2 chiudono il debito residuo; 3-5 implementano il challenger di costo; 6-7 abilitano il prossimo track.

Stato di partenza (riferimento, 2026-05-21)

Drift register — stato

ID	Item	Stato master
M1	Alpha158 naming → "Alpha158-inspired"	✅ Risolto (21b63af)
M2	Portfolio/risk docs vs config	⚠️ Parzialmente risolto
M3	Universe docs (54 ticker + crypto)	⚠️ Da verificare end-to-end
M4	Parkinson 1/(4 ln 2) canonical	✅ Risolto (21b63af)
M5		⚠️ Parziale — README ok,

ID	Item	Stato master
	EWM IC-Sharpe (no “rolling 100-day IR”)	aggregator docstring linea 7 ancora vecchia
M6	Orthogonality confidence shrinkage	✓ Risolto (21b63af)
M7	Sentiment source weighting nel daily	✓ Risolto (21b63af)
M8	Target engineering separato da inference	✗ Da fare
M9	Cost model “heuristic” non Almgren-Chriss	✓ Risolto (21b63af)
M10	Multivariate MC VaR	✓ Risolto (12276e8)
M11	Pickle hash sidecar (trusted_pickle_load)	✓ Risolto (21b63af)

TO-BE — allineamento con il mio report

Mia proposta	Stato master
5.4.2 Self-calibrating cost model	✓ Scelto come primo P2 (ADR-0003)
5.1 TFT/PatchTST, FinGPT/ RAG, switching SSM	📋 P2 challenger pool
5.2.1 MoE gating	📋 P2 (post-baseline)
5.3 HRP, robust opt, DCC-GARCH	📋 P2 (post-cost-track)
5.4.1 RL execution	📋 P3 (richiede fill history)
5.6.4 Observability (math trace)	📋 P1 (dashboard redesign)

Fase 0 — Sync del branch di lavoro con master

Obiettivo: allineare il branch `claude/codebase-algorithm-analysis-JmoBp` con `origin/master` per evitare conflitti quando si toccano file modificati di recente (aggregator, alpha158, pipeline, dashboard).

Pre-requisiti: working tree pulito.

Step: 1. `git fetch origin master` 2. `git rebase origin/master` sul branch corrente. Risolvere conflitti su docs/ se il PDF è coinvolto (preferire `--ours` per i file `codebase_analysis.{md,pdf}` aggiunti dal mio branch). 3. Verificare: `python -m pytest tests/test_council.py tests/test_features.py tests/test_artifact_governance.py -v` 4. `git push --force-with-lease origin claud/codebase-algorithm-analysis-JmoBp`

Done quando: `git log origin/master..HEAD --oneline` mostra solo il commit di docs analysis e tutti i test sopra passano.

Fase 1 — Chiusura drift residui (M2, M3, M5, M8)

Obiettivo: completare l'allineamento doc/config/code prima di iniziare ogni lavoro avanzato.

Task 1.1 — M5 (residuo): purgare “rolling 100-day IR” da docstrings interni

File: - `council/aggregator.py` — docstring di modulo (linea 7) ancora dice “rolling 100-day Information Ratio”. Sostituire con “EWM IC-Sharpe ($\text{halflife} \leq 20$)”. - Eventuali commenti residui in `aggregator.py:417,430` (“`ic_rolling_30d`”, “`sharpe_rolling_60d`”): mantenere come chiavi storiche di telemetria ma aggiungere docstring che chiarisca che il **calcolo** è EWM.

Verifica:

```
grep -rn "rolling 100-day" council/ docs/ README.md AGENTS.md
# expected: 0 hits
python -m pytest tests/test_council.py -v
```

Commit:

`docs(council): purge residual rolling-100-day IR wording (M5)`

Task 1.2 — M2: derivare doc risk da config/

File: - README.md sezione “Risk constraints” - docs/architecture-as-is-to-be-2026-05-21.md (aggiornare riga M2) - Nuovo helper `scripts/generate_risk_doc.py` che legge `config/runtime.env` + `council/portfolio.py` e stampa la tabella Markdown.

Approccio: invece di hard-codare i valori, usare un blocco markdown auto-generato delimitato da `<!-- BEGIN risk-table -->` / `<!-- END risk-table -->`, rigenerabile via `python scripts/generate_risk_doc.py --inplace`.

Verifica:

```
python scripts/generate_risk_doc.py --dry-run | diff - <(awk '/
    BEGIN risk-table/,/END risk-table/' README.md)
# expected: identical (or empty diff)
```

Commit: docs(risk): derive constraint table from config (M2)

Task 1.3 — M3: universo reale documentato

File: README.md, docs/architecture-as-is-to-be-2026-05-21.md
sezione M3.

Documentare: 54 ticker su 11 settori + BTCUSD/ETHUSD (crypto bucket separato), distinzione tra **research universe** (raw OHLCV) e **trading universe** (post load_universe_as_of() + history filter). Includere il diagramma del bucket selection.

Verifica: confronto manuale + `python -c "from data.pipeline import load_universe_as_of; print(len(load_universe_as_of()))"`
ritorna lo stesso numero documentato.

Commit: docs(universe): document real 54+2 universe and bucket contract (M3)

Task 1.4 — M8: separare target engineering da daily inference

File: - Nuovo docs/data-flow-daily-vs-training.md con 2 diagrammi Mermaid distinti (daily inference, offline training). - Aggiornare docs/architecture-as-is-to-be-2026-05-21.md M8 con link.

Verifica: review manuale; nessun cambio di codice in questo task.

Commit: docs(flow): separate daily-inference vs training-target diagrams (M8)

Done della Fase 1: drift register in architecture-as-is-to-be-2026-05-21.md ha tutte le righe M1-M11 marcate ; tabelle config rigenerabili da script.

Fase 2 — Telemetria fill: prerequisito per cost calibration

Obiettivo: raccogliere fill normalizzati in formato strutturato per alimentare la calibrazione di Fase 3.

Task 2.1 — Schema fill record

File: - Nuovo execution/fill_log.py con dataclass FillRecord: fill_id, order_id, ticker, side, qty, fill_price, decision_price, decision_ts, fill_ts, broker, venue, pipeline_run_id, config_hash, slippage_bps, commission_bps - data/operations/fills/{YYYY-MM}.parquet come storage partizionato per mese.

Verifica:

```
python -m pytest tests/test_fill_log.py -v # da creare
```

Commit: feat(execution): structured FillRecord + parquet log

Task 2.2 — Hook in OMS e Alpaca adapter

File: - execution/oms.py linea ~250 (dopo add_fill): chiamare fill_log.append(record) con decision_price letto da Order.created_at snapshot del prezzo di riferimento. - execution/alpaca_adapter.py:690-703 (paper_trades writer): far passare il record dallo stesso log.

Critico: decision_price deve essere il prezzo al momento della decisione (last close per market orders, limit price per limit). Aggiungere campo a Order dataclass se assente.

Verifica:

```
python -m pytest tests/test_oms.py tests/test_alpaca_adapter.py -v  
# Integration: paper trade end-to-end con assert su esistenza file  
fill log
```

Commit:

feat(oms): persist FillRecord on every fill (calibration input)

Task 2.3 — Backfill da paper_trades storici

File: scripts/backfill_fill_log.py che legge data/paper_trades/{date}.json esistenti e li converte in FillRecord.

Verifica: dopo run, pl.read_parquet("data/operations/fills/*.parquet").shape[0] ≥ numero righe in paper_trades.

Commit: chore(execution): backfill historical paper trades into fill log

Done della Fase 2: ogni nuovo fill produce automaticamente un FillRecord; storico esistente importato.

Fase 3 — Calibration engine (ADR-0003 Stage B core)

Obiettivo: modulo che legge FillRecord e produce cost_calibration.json con kappa_slippage_bps per ticker/tier.

Task 3.1 — Implementation Shortfall computation

File: nuovo council/cost_calibration.py.

Formula (da ADR):

```
IS_bps = 10_000 * (fill_price - decision_price) / decision_price *  
sign(side)
```

API:

```
class CostCalibrator:  
    def __init__(self, fills_path: Path, min_fills: int = 30): ...  
    def compute_kappa(self, tier: str | None = None) -> dict[str,  
        float]: ...  
    def write_calibration(self, out_path: Path) -> str: ... #  
        returns sha256
```

Sample logic: - Per ticker: rolling median IS_bps su ultimi N fills.
- Per tier (mega/large/mid/crypto): aggregate dei ticker del tier. -
Se $n < \text{min_fills}$ → escludere (fallback al lookup statico).

Verifica:

```
python -m pytest tests/test_cost_calibration.py -v  
# Test: fixtures con fills sintetici → kappa atteso; insufficient  
sample → key assente
```

Commit: feat(council): cost calibration engine (rolling median IS_bps)

Task 3.2 — Manifest + sidecar

File: stesso modulo. Output data/operations/cost_calibration.json + .manifest (JSON con pipeline_run_id, config_hash, fill_sample_count, calibration_window_end, sha256_of_json).

Riusare council/pickle_security.py per pattern di hash sidecar (anche se è JSON, mantenere convenzione SHA-256 sidecar per audit consistency).

Verifica: test test_cost_calibration_manifest.py che verifica round-trip write→read→hash-check.

Commit: feat(council): cost calibration artifact with hash sidecar

Task 3.3 — Job notturno

File: - Nuovo Dagster asset in data/pipeline.py (Layer 4 estensione): cost_calibration_artifact con RetryPolicy(max_retries=2, delay=60). - Schedule cron 0 23 * * * (23:00 UTC, dopo close US). - Dipendenze: dipende da daily_orders (per pipeline_run_id) e da fill_log non-Dagster (lettura diretta filesystem).

Verifica:

```
dagster asset materialize -f data/pipeline.py --select
    cost_calibration_artifact
# inspect: data/operations/cost_calibration.json esistente, valid
JSON
```

Commit: feat(pipeline): nightly cost_calibration_artifact asset

Done della Fase 3: girando il pipeline si produce un file di calibrazione versionato e verificabile.

Fase 4 — Wire calibration in TransactionCostModel

Obiettivo: far consumare la calibrazione al modello di costo che entra in CVXPY + backtest, con rollback.

Task 4.1 — Blend lookup + calibrato

File: council/transaction_costs.py.

Modifica TransactionCostModel.from_env():

```
calib_path = os.getenv("MLCOUNCIL_COST_CALIBRATION_PATH",
                       "data/operations/cost_calibration.json")
if exists(calib_path) and manifest_verifies(calib_path):
    kappa_calibrated = load_calibration(calib_path)
    alpha = min(1.0, fill_count / CONFIDENCE_FLOOR) # 0..1
    slippage_bps[ticker] = (1 - alpha) * lookup[ticker] + alpha *
        kappa_calibrated[ticker]
else:
    slippage_bps = lookup # fallback statico
```

Critico: comportamento fail-safe — se manifest non verifica, log WARNING e usa lookup statico (NON sollevare eccezione).

Verifica:

```
python -m pytest tests/test_transaction_costs.py -v
# nuovi test: con calib file, senza calib file, con calib hash
mismatch
```

Commit: feat(costs): blend calibrated kappa with static lookup

Task 4.2 — Lineage in order/execution log

File: execution/oms.py, data/pipeline.py (daily_orders asset).

Aggiungere campo cost_calibration_version (SHA-256 del manifest attivo al momento della decisione) a Order dataclass e ai parquet daily_orders.

Verifica: ispezionare data/orders/{date}.parquet e confermare presenza colonna.

Commit: feat(lineage): tag orders with cost_calibration_version

Task 4.3 — Backtest costs walk-forward

File: backtest/runner.py, backtest/report.py.

Estendere _compute_stats per produrre due metriche separate: net_sharpe_static_costs e net_sharpe_calibrated_costs. Il delta è l'evidenza promotion per Fase 5.

Verifica:

```
python scripts/run_strategy_backtest.py --cost-mode=both --output
data/results/cost_ab.json
python -m pytest tests/test_backtest_validation.py -v
```

Commit: feat(backtest): A/B static vs calibrated cost reporting

Done della Fase 4: un singolo run di backtest produce sia metriche static che calibrated; live trading usa calibrated con rollback automatico.

Fase 5 — Validation, alerting, promotion

Obiettivo: governance per evitare che una calibrazione tossica entri in produzione.

Task 5.1 — Divergence alert

File: council/monitor.py (nuovo check cost_calibration_divergence_check).

Regola: se $|\text{kappa_calibrated} - \text{kappa_lookup}| > 5$ bps per ≥ 5 sessioni consecutive sullo stesso tier → alert WARNING. Soglia: critical se > 15 bps.

File config: aggiungere chiave in config/monitoring.yaml (creare se non esiste).

Verifica: python -m pytest tests/test_monitor.py -k cost_calibration -v

Commit: feat(monitor): cost calibration divergence alert

Task 5.2 — Champion/challenger gate (costo, non modello)

File: backtest/validation.py.

Nuova funzione validate_cost_calibration_promotion() con criteri: - net Sharpe calibrated $\geq$ net Sharpe static - 0.1 (no regressione marcata) - turnover delta entro $\pm 10\%$ - implementation shortfall mediano dei fill $<$ lookup - fill_sample_count ≥ 30 per tier promosso

Se fallisce → MLCOUNCIL_COST_CALIBRATION_PATH="" automatico (revert a statico) via env override scritto in config/runtime_override.env.

Verifica: python -m pytest tests/test_validation.py -k cost -v

Commit: feat(validation): cost calibration promotion gate

Task 5.3 — Baseline post-cost-track

File: docs/baselines/YYYY-MM-DD-cost-calibrated-baseline.md (segue il template di docs/baselines/2026-05-21-clean-baseline.md).

Catturare: Sharpe lordo/netto, turnover, IS medio, breach count, runtime. Confronto side-by-side col clean baseline.

Commit: docs(baseline): cost-calibrated baseline measurement

Done della Fase 5: track #1 P2 chiuso con evidenza A/B; sistema può promuovere o rollback.

Fase 6 — Dashboard math-trace MVP (P1 observability)

Obiettivo: rendere ogni decisione tracciabile dall'UI senza redesign completo.

Task 6.1 — Constraint waterfall per il portfolio

File: `dashboard/charts.py` nuovo grafico `optimizer_waterfall(date)`.

Mostrare, per una data: 1. Pesi greedy $\alpha * \text{multiplier}$ (pre-CVXPY) 2. Pesi dopo budget constraint 3. Pesi dopo position cap 4. Pesi dopo sector cap 5. Pesi dopo turnover cap 6. Pesi dopo vol cap 7. Pesi finali

Ogni step = barra con delta vs step precedente, evidenziando vincoli binding.

File: `council/portfolio.py` deve già esporre `optimization_diagnostics` dict (verificare; se assente, esporlo).

Verifica: apri Streamlit, naviga a `/portfolio?date=YYYY-MM-DD`, conferma rendering.

Commit: `feat(dashboard): optimizer constraint waterfall chart`

Task 6.2 — Council attribution trace

File: `dashboard/data_loader.py:363` usa già `trusted_pickle_load` sul `aggregator_state`. Estendere la pagina dashboard per mostrare per ogni data: - `weights_log[date]["weights"]` - `effective_weight_sum` (M6) - `ortho_applied` flag - IC e Sharpe rolling per modello

Commit: `feat(dashboard): council attribution math-trace panel`

Task 6.3 — Fill quality panel

File: nuova pagina `dashboard/pages/fill_quality.py`.

Mostrare per ticker: IS mediano, slippage realized vs assunto (lookup), trend `kappa_calibrated` nel tempo, sample size.

Commit: `feat(dashboard): fill quality and cost calibration panel`

Done della Fase 6: ogni claim del backtest report è verificabile cliccando una data sul dashboard.

Fase 7 — Selezione del prossimo track avanzato

Obiettivo: decidere se il secondo P2 è HRP-soft-prior, robust optimization, o dashboard product redesign.

Task 7.1 — Mini-spike HRP

File: nuovo notebooks/2026-XX-XX-hrp-spike.ipynb (oppure script).

Implementare HRP (López de Prado) standalone su ultimi 90 giorni di returns, confrontare allocazione vs CVXPY MV su 5 date sample. Misura: condition number della matrice di covarianza usata, tail-risk dei pesi.

Deliverable: ADR docs/adr/YYYY-MM-DD-hrp-soft-prior.md con go/no-go.

Task 7.2 — Mini-spike Robust Optimization

File: nuovo notebook/script con CVXPY robust:

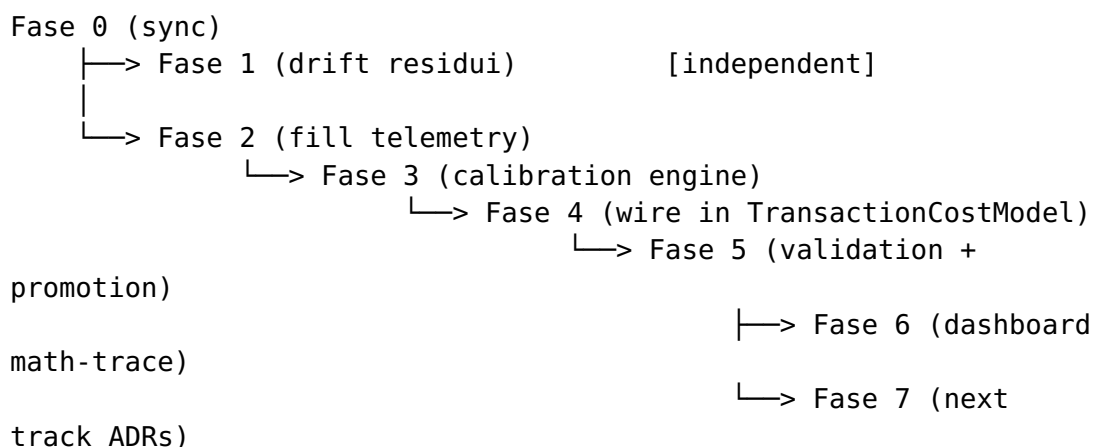
$\max_w \alpha^T w - \kappa \sqrt{w^T \Sigma w}$ $\max_w \hat{\alpha}^T w - \kappa \sqrt{w^T \Sigma w}$ calibrare κ su uno scan. Confronto rispetto al MV su 12 mesi walk-forward.

Deliverable: ADR analogo.

Task 7.3 — Decision review

Aprire ExitPlanMode con l'utente per scegliere: HRP / Robust / Dashboard redesign / TFT spike.

Riassunto dipendenze tra fasi



Le Fasi 1 e 2 possono essere eseguite **in parallelo** da due agenti.

Acceptance criteria globale

- Tutti i test verdi: `python -m pytest tests/ -v --timeout=120`
- Drift register architecture-as-is-to-be-2026-05-21.md 11/11 
- File `data/operations/cost_calibration.json` viene rigenerato ogni notte
- Backtest report mostra `net_sharpe_static` e `net_sharpe_calibrated`
- Dashboard renderizza waterfall + council attribution + fill quality
- Due ADR esistono per selezione del prossimo track

Comandi rapidi per l'agente

Setup

```
git fetch origin master && git rebase origin/master
```

Test runner (rapido)

```
python -m pytest tests/test_council.py tests/test_features.py  
tests/test_artifact_governance.py -v
```

Test runner (completo)

```
python -m pytest tests/ -v --timeout=120
```

Backtest A/B costi

```
python scripts/run_strategy_backtest.py --cost-mode=both
```

Materializza calibration

```
dagster asset materialize -f data/pipeline.py --select  
cost_calibration_artifact
```

Dashboard locale

```
streamlit run dashboard/app.py
```

Note operative per l'agente

1. **Ogni fase = 1 PR** con commit atomici (uno per task).
2. **Mai saltare la verifica:** i comandi di test elencati per ogni task sono il gate.
3. **Rollback safe by design:** ogni step di Fase 3-4 deve mantenere il fallback statico se la calibrazione fallisce.
4. **No model retrain in queste fasi:** zero modifiche a LGBM, FinBERT, HMM. Lo scopo è onestà di costo e osservabilità.
5. **Doc-first ai mismatch:** se un task richiede modificare codice e doc, prima il test, poi il codice, poi la doc.

6. **Hash sidecar obbligatorio:** ogni nuovo artifact pickle/JSON in data/operations/ deve avere .hash sibling — riutare council/pickle_security.py.